

Объектно-ориентированное программирование

Понять объектный подход: класс как описание, объект как конкретный экземпляр, атрибуты как данные и методы как поведение.

DAY 18

CLASS

object

attribute

method

constructor

ЦЕЛЬ ДНЯ

Что ты сможешь после этого дня

Конкретные действия вместо списка терминов.

- 01 Понять объектный подход: класс как описание, объект как конкретный экземпляр, атрибуты как данные и методы как поведение.
- 02 Объяснить минимум 5 терминов темы своими словами.
- 03 Прочитать показанный пример кода и предсказать результат.
- 04 Самостоятельно решить похожую задачу после разбора.

СВЯЗЬ

Новая тема не появляется из ниоткуда

Связываем сегодняшний шаг с уже построенной картой курса.

ВЧЕРА

Файлы, JSON и CSV

Что уже есть в нашей модели.



СЕГОДНЯ

Объектно-ориентированное программирование

Вчера: Файлы, JSON и CSV. Сегодня мы добавляем следующий слой: объектно-ориентированное программирование.

ЗАЧЕМ

Зачем мне это?

Каждая тема решает конкретную проблему.

ПРОБЛЕМА → ПОЛЬЗА

ООП помогает объединять связанные данные и поведение и удобно моделировать сущности предметной области.

Вопрос себе

Какую реальную проблему решает сегодняшняя тема?

Ответь до следующего слайда.

Сначала — человеческий образ

Потом обязательно связываем его с настоящей техникой.

БЫТОВОЙ ОБРАЗ

Класс — чертёж дома, объект — конкретный дом, созданный по чертежу; атрибуты — его свойства, методы — действия, которые объект умеет выполнять.

ПЕРЕВОД В IT

class	описание структуры и поведения объектов
object	конкретный экземпляр класса
attribute	данные, принадлежащие объекту

Слой 1: class • object • attribute

Сначала смысл, затем профессиональный термин.

**class**

описание структуры и поведения объектов

**object**

конкретный экземпляр класса

**attribute**

данные, принадлежащие объекту

Слой 2: method • constructor • inheritance

Сначала смысл, затем профессиональный термин.

**method**

функция, определённая внутри класса

**constructor**

логика начальной настройки нового объекта

**inheritance**

создание класса на основе поведения другого класса

Слой 3: instance • encapsulation

Сначала смысл, затем профессиональный термин.

instance

конкретный объект, созданный из класса

encapsulation

объединение данных и поведения внутри объекта

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Слой 4: polymorphism • super

Сначала смысл, затем профессиональный термин.

polymorphism

единый интерфейс для объектов разных типов

super

обращение к реализации родительского класса

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Как это работает по шагам

Проследи движение слева направо.



Проговори эту цепочку своими словами, не подсматривая в текст.

Первый минимальный пример

Смотрим только на одну основную идею.



day18_step1

```
class Course:
    def __init__(self, title):
        self.title = title

    def describe(self):
        return f"Курс: {self.title}"

course = Course("Python")
print(course.describe())
```

ЧТО ИСКАТЬ

- Где данные?
- Где действие?
- Где результат?
- Что изменится, если заменить одно значение?

Разбираем пример построчно

Код не должен оставаться магической конструкцией.

```
class Course:
```

Создаём описание нового типа объектов — класс.

```
def __init__(self, title):
```

Создаём функцию и даём ей имя; код внутри выполнится при вызове.

```
self.title = title
```

Сохраняем значение или результат вычисления под понятным именем.

```
def describe(self):
```

Создаём функцию и даём ей имя; код внутри выполнится при вызове.

```
return f"Курс: {self.title}"
```

Python выполняет эту строку сверху вниз.

РЕЗУЛЬТАТ

Код → выполнение → результат

Всегда связываем запись с тем, что реально получилось.



до запуска

```
class Course:
    def __init__(self, title):
        self.title = title

    def describe(self):
        return f"Курс: {self.title}"

course = Course("Python")
print(course.describe())
```

РЕЗУЛЬТАТ

Курс: Python

ИЗМЕНИЛИ

Теперь изменим пример

Сначала предскажи результат. Не запускай код сразу.

```
class Counter:
    def __init__(self):
        self.value = 0
    def increment(self):
        self.value += 1

c = Counter()
c.increment()
print(c.value)
```

changed



**Что появится в
результате?**

Сформулируй ответ словами.

ОТВЕТ

Сверяем прогноз

Не просто смотри ответ — объясни, почему он именно такой.

РЕЗУЛЬТАТ

1

ПОЧЕМУ

Python/система выполняет шаги в записанном порядке. Изменённые данные или действие дают новый результат.

Сравни с первым примером: что изменилось?

Как писать неправильно

Ошибка — отдельный учебный материал, а не повод её скрывать.

```
class User:  
    def __init__(name):  
        name = name
```

wrong

СТОП

**Что здесь может сломаться
или ввести нас в
заблуждение?**

Сначала найди проблему сам.

Почему возникает проблема

Переводим техническую причину на обычный язык.

Метод экземпляра должен получать `self` первым параметром, а значение нужно сохранить в `self.name`, иначе объект не запомнит его.

Алгоритм отладки: симптом → конкретная строка/шаг → причина → минимальное исправление → повторная проверка.

Исправленный вариант

Меняем только то, что действительно было причиной.

```
class User:  
    def __init__(self, name):  
        self.name = name
```

fixed

ПРАВИЛО

Не переписывай всё решение. Найди минимальное осмысленное исправление.

Что изменилось после исправления?

Ответь без запуска.

- 1 Исчезла ли исходная причина проблемы?
- 2 Какой результат/поведение теперь ожидаем?
- 3 Как проверить это одним маленьким примером?

Сначала увидим полностью разобранный пример

Только после него будет самостоятельная практика.

ЗАДАЧА

Создать Product с ценой и методом `final_price(discount)`, который возвращает цену после скидки.

Не начинаем с кода. Сначала переводим задачу на человеческий язык.

КАК ДУМАТЬ

Как рассуждает программист

Разбиваем большую формулировку на маленькие действия.

- 1 **Класс хранит name и price.**
- 2 `__init__` получает данные нового объекта.
- 3 Метод получает discount.
- 4 Использует `self.price`.
- 5 Возвращает вычисленную цену.

Шаг 1: собираем основу

Не показываем 20 строк сразу — строим решение частями.



solution_part1

```
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price

    def final_price(self, discount):
```

Проверь

- Какие данные уже есть?
- Какой шаг плана реализован?
- Что ещё осталось добавить?

ИТОГОВЫЙ КОД

Шаг 2: итоговое решение

Теперь видим программу/запрос целиком.



solution_final

```
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price

    def final_price(self, discount):
        return self.price * (1 - discount)

p = Product("Book", 20)
print(p.final_price(0.1))
```

Проверяем результат

Сравниваем то, что ожидали, с тем, что получили.

РЕЗУЛЬТАТ

18.0

ГЛАВНОЕ

Задача была решена не потому, что мы «знали готовый код», а потому что сначала построили понятный план.

ПОПРОБУЙ САМ

Практика 1

Задача очень похожа на разобрannую — но теперь первый ход за тобой.

Сделай класс Counter с `increment()` и `reset()`.

Сначала выпиши 2–5 шагов решения словами.

ПОДСКАЗКА

Подсказка — только направление

Попробуй ещё раз до просмотра готового решения.

Состояние хранится в `self.value`; методы меняют именно этот атрибут.

Вопрос себе: какие конструкции сегодняшнего дня нужны, а какие пока лишние?

Один из правильных вариантов

Сравни логику, а не количество строк.



practice_solution

```
class Counter:
    def __init__(self):
        self.value = 0
    def increment(self):
        self.value += 1
    def reset(self):
        self.value = 0

c = Counter(); c.increment(); c.reset(); print(c.value)
```

Ожидаемый результат

Если твой код устроен иначе, но даёт правильное и понятное решение — это нормально.

РЕЗУЛЬТАТ

0

Сравни

- Результат
- Понятность имён
- Лишние шаги
- Обработку ошибок

Практика 2 — мини-проект

Соединяем несколько элементов дня в одну маленькую закончённую задачу.

ЗАДАНИЕ

Сделай класс `Student` с `name`, `scores` и методом `average()`.

Попробуй сначала без подсказки. Разрешено подсматривать в сегодняшние примеры.

Разбор мини-проекта

Вот один рабочий путь.



mini_project

```
class Student:
    def __init__(self, name, scores):
        self.name = name
        self.scores = scores
    def average(self):
        return sum(self.scores) / len(self.scores)

s = Student("Anna", [5, 4, 5])
print(round(s.average(), 2))
```

РЕЗУЛЬТАТ ПРОЕКТА

Что должно получиться

Связываем итог с задачей и проверяем смысл.

РЕЗУЛЬТАТ

4.67

Объясни каждую важную строку/команду своими словами. Если не можешь — вернись к конкретному месту, а не перечитывай весь день.

Четыре ловушки новичка

Лучше увидеть их здесь, чем случайно закрепить как привычку.

01 Путать класс и объект.

02 Забывать `self` в методах экземпляра.

03 Хранить состояние в локальной переменной вместо `self.*`.

04 Использовать наследование там, где достаточно простой композиции.

Проверь себя: 1–2

Не листай дальше, пока не сформулируешь ответ.

1

Что такое «class» и какую проблему это понятие помогает описать?

2

Чем «class» отличается от «method»? Объясни без копирования определения.

Проверь себя: 3–4

Здесь проверяем понимание кода и процесса мышления.

- 3 Какой результат даёт первый минимальный пример и почему?
- 4 Какую ошибку/риск мы разобрали сегодня и какое минимальное исправление нужно?

ОТВЕТЫ

Ответы к вопросам 1–2

Формулировка может отличаться — важен смысл.

1 **class — описание структуры и поведения объектов.**

2 **class: описание структуры и поведения объектов. method: функция, определённая внутри класса.**

Ответы к вопросам 3–4

Сверяем причинно-следственную связь.

3 Курс: Python

4 Метод экземпляра должен получать `self` первым параметром, а значение нужно сохранить в `self.name`, иначе объект не запомнит его.

ИТОГ

Сегодня мы научились

Не перечисляем слова — фиксируем новые способности.

- ✓ Понять объектный подход: класс как описание, объект как конкретный экземпляр, атрибуты как данные и методы как поведение.
- ✓ Читать и объяснять пример темы «Объектно-ориентированное программирование».
- ✓ Находить типичную ошибку и формулировать причину.
- ✓ Решать похожую задачу по шагам, а не угадывать готовый код.

ДАЛЬШЕ

Что будет завтра

Связываем сегодняшний результат со следующим шагом.

Git и совместная работа

Завтра: Git и совместная работа. Сегодняшняя модель станет опорой для следующего шага.

Сегодняшняя тема не заканчивается здесь — она станет инструментом в следующем дне.